

Лабораторная работа №5

**Дискреционное разграничение прав в Linux. Исследование
влияния дополнительных атрибутов**

Павлова Татьяна Юрьевна

Содержание

1	Цель работы	5
2	Теоретическое введение	6
3	Выполнение лабораторной работы	8
4	Выводы	15
5	Список литературы	16

Список иллюстраций

3.1	gcc -v	8
3.2	Создание файла	8
3.3	Содержимое файла	9
3.4	Компиляция файла	9
3.5	Создание и компиляция файла	9
3.6	Содержимое файла	10
3.7	Смена владельца файла и прав доступа к файлу	10
3.8	Запуск файла	10
3.9	Содержимое файла	11
3.10	Смена владельца файла и прав доступа к файлу	11
3.11	Попытка прочесть содержимое файла	11
3.12	Попытка прочесть содержимое файла программой	12
3.13	Попытка прочесть содержимое файла программой	12
3.14	Проверка атрибутов директории tmp	12
3.15	Создание файла, изменение прав доступа	13
3.16	Попытка чтения файла	13
3.17	Смена атрибутов файла	13
3.18	Повтор предыдущих действий	14
3.19	Изменение атрибутов	14

Список таблиц

1 Цель работы

Целью данной лабораторной работы является изучение изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

2 Теоретическое введение

1. Дополнительные атрибуты файлов Linux

В Linux существует три основных вида прав — право на чтение (read), запись (write) и выполнение (execute), а также три категории пользователей, к которым они могут применяться — владелец файла (user), группа владельца (group) и все остальные (others). Но, кроме прав чтения, выполнения и записи, есть еще три дополнительных атрибута.

Sticky bit

Используется в основном для каталогов, чтобы защитить в них файлы. В такой каталог может писать любой пользователь. Но, из такой директории пользователь может удалить только те файлы, владельцем которых он является. Примером может служить директория /tmp, в которой запись открыта для всех пользователей, но нежелательно удаление чужих файлов.

SUID (Set User ID)

Атрибут исполняемого файла, позволяющий запустить его с правами владельца. В Linux приложение запускается с правами пользователя, запустившего указанное приложение. Это обеспечивает дополнительную безопасность т.к. процесс с правами пользователя не сможет получить доступ к важным системным файлам, которые принадлежат пользователю root.

SGID (Set Group ID)

Аналогичен suid, но относится к группе. Если установить sgid для каталога, то все файлы созданные в нем, при запуске будут принимать идентификатор группы

каталога, а не группы владельца, который создал файл в этом каталоге.

Обозначение атрибутов sticky, suid, sgid

Специальные права используются довольно редко, поэтому при выводе программы `ls -l` символ, обозначающий указанные атрибуты, закрывает символ стандартных прав доступа.

Пример: `rwsrwsrwt`

где первая `s` — это `suid`, вторая `s` — это `sgid`, а последняя `t` — это `sticky bit`

В приведенном примере не понятно, `rwt` — это `rw-` или `rwX`? Определить это просто. Если `t` маленькое, значит `x` установлен. Если `T` большое, значит `x` не установлен. То же самое правило распространяется и на `s`.

В числовом эквиваленте данные атрибуты определяются первым символом при четырехзначном обозначении (который часто опускается при назначении прав), например в правах `1777` — символ `1` обозначает `sticky bit`.

2. Компилятор GCC

`GCC` - это свободно доступный оптимизирующий компилятор для языков `C`, `C++`. Собственно программа `gcc` это некоторая надстройка над группой компиляторов, которая способна анализировать имена файлов, передаваемые ей в качестве аргументов, и определять, какие действия необходимо выполнить. Файлы с расширением `.cc` или `.C` рассматриваются, как файлы на языке `C++`, файлы с расширением `.c` как программы на языке `C`, а файлы с расширением `.o` считаются объектными `[gcc]`.

3 Выполнение лабораторной работы

Для лабораторной работы необходимо проверить, установлен ли компилятор gcc, команда `gcc -v` позволяет это сделать. Также осуществляется отключение системы запретов с помощью `setenforce 0` (рис. 1).

```
lguest@localhost tanyal$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/14/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgc-n-amdhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-redhat-linux
Configured with: ../configure --enable-bootstrap --enable-languages=c,c++,fortran,lto --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bug-url=https://bugs.rockylinux.org --enable-shared --enable-threads=posix --enable-checking=release --with-system-zlib --enable-_cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-gcc-major-version-only --enable-plugin --enable-initfini-array --enable-libstdc++-backtrace --with-libstdc++-zoneinfo=/usr/share/zoneinfo --without-isl --enable-host-pie --enable-host-bind-now --enable-multilib --with-linker-hash-style=gnu --enable-offload-targets=nvptx-none,amdgc-n-amdhsa --enable-offload-defaulted --without-cuda-driver --enable-gnu-indirect-function --enable-cet --with-tune=generic --with-arch=x86-64-v3 --with-arch32=x86-64 --build=x86_64-redhat-linux --with-build-config=bootstrap-lto --enable-link-serialization=1
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 14.3.1 20251022 (Red Hat 14.3.1-4) (GCC)
lguest@localhost tanyal$
```

Рисунок 3.1: gcc -v

Осуществляется вход от имени пользователя guest

Создание файла `simpleid.c` и запись в файл кода (рис. 2)

```
[lguest@localhost ~]$ touch simpleid.c
```

Рисунок 3.2: Создание файла

Содержимое файла выглядит следующи образом (рис. 3)


```

GNU nano 8.1                                simpleid.c
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main(){
    uid_t uid = geteuid();
    gid_t gid = getegid();
    printf("uid=%d, gid=%d\n", uid, gid);
    return 0;
}

```

Рисунок 3.3: Содержимое файла

Компилирую файл, проверяю, что он скомпилировался (рис. 4)

```

[guest@localhost ~]$ gcc simpleid.c -o simpleid
[guest@localhost ~]$ ./simpleid
uid=1001, gid=1001
[guest@localhost ~]$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[guest@localhost ~]$

```

Рисунок 3.4: Компиляция файла

Запускаю исполняемый файл. В выводе файла выписаны номера пользователя и групп, от вывода при вводе `id`, они отличаются только тем, что информации меньше

Создание, запись в файл и компиляция файла `simpleid2.c`. Запуск программы (рис. 5)

```

GNU nano 8.1                                simpleid2.c
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main(){
    uid_t real_uid = geteuid();
    uid_t e_uid = geteuid();

    gid_t real_gid = getegid();
    gid_t e_gid = getegid();
    printf("e_uid=%d e_gid=%d\n", e_uid, e_gid);
    printf("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
    return 0;
}

```

Рисунок 3.5: Создание и компиляция файла

```
[guest@localhost ~]$ gcc simpleid2.c -o simpleid2
[guest@localhost ~]$ ./simpleid2
e_uid=1001 e_gid=1001
real_uid=1001, real_gid=1001
```

Рисунок 3.6: Содержимое файла

С помощью `chown` изменяю владельца файла на суперпользователя, с помощью `chmod` изменяю права доступа (рис. 7)

```
[guest@localhost ~]$ sudo ls -l simpleid2
[sudo] password for guest:
guest is not in the sudoers file.
[guest@localhost ~]$ sudo ls -l /home/guest/simpleid2
[sudo] password for guest:
guest is not in the sudoers file.
[guest@localhost ~]$ ls -l /home/guest/simpleid2
-rwsr-xr-x. 1 root guest 16872 Aug 23 23:23 /home/guest/simpleid2
```

Рисунок 3.7: Смена владельца файла и прав доступа к файлу

Сравнение вывода программы и команды `id`, наша команда снова вывела только ограниченное количество информации (рис. 8)

```
[guest@localhost ~]$ ./simpleid2
e_uid=0 e_gid=1001
real_uid=0, real_gid=1001
[guest@localhost ~]$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Рисунок 3.8: Запуск файла

Создание и компиляция файла `readfile.c` (рис. 9)



```

GNU nano 8.1                                readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

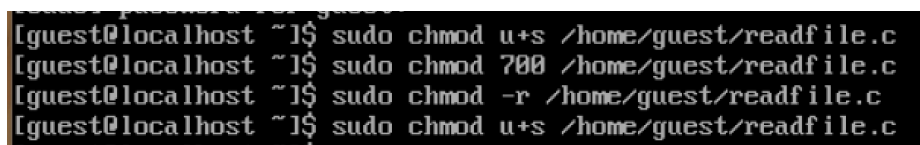
int main(int argc, char* argv[]) {
    unsigned char buffer[16];
    size_t bytes_read;
    int i;

    int fd = open(argv[1], O_RDONLY);
    do {
        bytes_read = read(fd, buffer, sizeof(buffer));
        for(i=0; i<bytes_read; i++) printf("%c", buffer[i]);
    } while(bytes_read == sizeof(buffer));
    close(fd);
    return 0;
}

```

Рисунок 3.9: Содержимое файла

Снова от имени суперпользователя меняю владельца файла readfile. Далее меняю права доступа так, чтобы пользователь guest не смог прочесть содержимое файла (рис. 10)



```

[guest@localhost ~]$ sudo chmod u+s /home/guest/readfile.c
[guest@localhost ~]$ sudo chmod 700 /home/guest/readfile.c
[guest@localhost ~]$ sudo chmod -r /home/guest/readfile.c
[guest@localhost ~]$ sudo chmod u+s /home/guest/readfile.c

```

Рисунок 3.10: Смена владельца файла и прав доступа к файлу

Проверка прочесть файл от имени пользователя guest. Прочесть файл не удастся (рис. 11)



```

[guest@localhost ~]$ cat readfile.c
cat: readfile.c: Permission denied

```

Рисунок 3.11: Попытка прочесть содержимое файла

Попытка прочесть тот же файл с помощью программы readfile, в ответ получаем «отказано в доступе» (рис. 12)

```
[guest@localhost ~]$ ./readfile readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main(int argc, char* argv[]){
    unsigned char buffer[16];
    size_t bytes_read;
    int i;

    int fd = open(argv[1], O_RDONLY);
    do{
        bytes_read = read(fd, buffer, sizeof(buffer));
        for(i=0; i<bytes_read; i++)printf("%c", buffer[i]);
    }
    while(bytes_read == sizeof(buffer));
    close(fd);
    return 0;
}
```

Рисунок 3.12: Попытка прочесть содержимое файла программой

Попытка прочесть файл `\etc\shadow` с помощью программы, все еще получаем отказ в доступе (рис. 13)

```
[guest@localhost ~]$ ./readfile /etc/shadow
[15536.441492] readfile[3844]: segfault at 7ffd17f9a000 ip 00000000004011ad sp 0
0007ffd17f992f0 error 4 in readfile[11ad,401000+1000] likely on CPU 10 (core 10,
socket 0)
[15536.442364] Code: 45 f8 48 8d 4d e0 8b 45 f8 ba 10 00 00 00 48 89 ce 89 c7 e8
b5 fe ff ff 48 89 45 f0 c7 45 fc 00 00 00 00 eb 18 8b 45 fc 48 98 <0f> b6 44 05
e0 0f b6 c0 89 c7 e8 74 fe ff ff 83 45 fc 01 8b 45 fc
Segmentation fault (core dumped)
```

Рисунок 3.13: Попытка прочесть содержимое файла программой

Проверяем папку `tmp` на наличие атрибута `Sticky`, т.к. в выводе есть буква `t`, то атрибут установлен (рис. 14)

```
[guest@localhost ~]$ sudo ls -l / | grep tmp
drwxrwxrwt. 10 root root 4096 Aug 23 23:58 tmp
```

Рисунок 3.14: Проверка атрибутов директории `tmp`

От имени пользователя `guest` создаю файл с текстом, добавляю права на чтение и запись для других пользователей (рис. 15)

```
[guest0@localhost ~]$ echo "test" > /tmp/file01.txt
[guest0@localhost ~]$ ls
dir1 readfile readfile.c simpleid simpleid2 simpleid2.c simpleid.c
[guest0@localhost ~]$ chmod o+rw /tmp/file01.txt
[guest0@localhost ~]$ ls -l /tmp/file01.txt
-rw-r--rw-. 1 guest guest 5 Aug 24 00:08 /tmp/file01.txt
```

Рисунок 3.15: Создание файла, изменение прав доступа

Вхожу в систему от имени пользователя guest2, от его имени могу прочитать файл file01.txt, но перезаписать информацию в нем не могу. Также невозможно добавить в файл file01.txt новую информацию от имени пользователя guest2. (рис. 16)

```
[guest0@localhost ~]$ su guest2
Password:
[guest2@localhost guest1]$ ls
dir1 readfile readfile.c simpleid simpleid2 simpleid2.c simpleid.c
[guest2@localhost guest1]$ cd
[guest2@localhost ~]$ cat /tmp/file01.txt
test
[guest2@localhost ~]$ echo "test2" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@localhost ~]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? no
[guest2@localhost ~]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? yes
rm: cannot remove '/tmp/file01.txt': Operation not permitted
[guest2@localhost ~]$
```

Рисунок 3.16: Попытка чтения файла

От имени суперпользователя снимаем с директории атрибут Sticky (рис. 17)

```
[guest2@localhost ~]$ su -
Password:
Last login: Sun Aug 23 23:51:12 MSK 2026 on tty1
[root@localhost ~]# chmod -t /tmp
[root@localhost ~]# exit
logout
[guest2@localhost ~]$ ls -l / | grep tmp
drwxrwxrwx. 10 root root 4096 Aug 24 00:15 tmp
```

Рисунок 3.17: Смена атрибутов файла

Далее был выполнен повтор предыдущих действий. По результатам без Sticky-бита запись в файл и дозапись в файл осталась невозможной, зато удаление файла прошло успешно (рис. 18)

```
[guest2@localhost ~]$ cat /tmp/file01.txt
test
[guest2@localhost ~]$ echo "test2" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@localhost ~]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
[guest2@localhost ~]$
```

Рисунок 3.18: Повтор предыдущих действий

Возвращение директории tmp атрибута t от имени суперпользователя (рис. 19)

```
[tanya@localhost ~]$ su -
Password:
Last login: Mon Aug 24 00:14:49 MSK 2026 on tty1
[root@localhost ~]# chmod +t /tmp
[root@localhost ~]# exit
logout
```

Рисунок 3.19: Изменение атрибутов

4 Выводы

В ходе выполнения данной лабораторной работы, я изучила изменение идентификаторов, применение SetUID- и Sticky-битов. Получила практические навыки работы в консоли с дополнительными атрибутами. Рассмотрела работу механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

5 Список литературы

[0] Методические материалы курса